

Design and Verification Report: VHDL Elevator Control System

Digital Logic Design Unit

December 20, 2025

1 Introduction

The purpose of this project is to design a robust **Elevator Control System** capable of managing an 8-floor building (0 to 7). The system utilizes a Finite State Machine (FSM) to handle floor requests, motor control, door operations, and emergency safety protocols.

2 Design Flow

The design process followed a modular hardware development lifecycle:

- **Specification:** Defining the input/output ports. Inputs include **CALL** (target floor), **STOP** (emergency), and synchronization signals. Outputs control the **MOTOR** and **DOOR** status.
- **FSM Synthesis:** Implementing a hybrid Moore/Mealy machine where the state transitions depend on floor comparisons and the **STOP** signal.
- **RTL Implementation:** Coding the logic in VHDL using synchronous sequential processes for state registers and floor tracking, and combinatorial processes for next-state logic.
- **Verification:** Simulating the RTL against a testbench to validate path coverage (Up, Down, Stop, and Door Timing).

3 Functional Working (RTL)

The core of the system is the `Elevator_Control` entity. It operates based on five distinct states.

3.1 Finite State Machine (FSM) States

- **IDLE:** The elevator is stationary. It latches the **CALL** input into a `latched_target` register and decides whether to move up or down.
- **GOING_UP / GOING_DOWN:** The system activates `MOTOR_UP` or `MOTOR_DOWN`. The `curr_floor` signal increments or decrements on each clock cycle until it matches the target.

- **OPEN_DOOR:** Upon reaching the target floor, the system enters this state for exactly 5 clock cycles, managed by a 4-bit `timer`.
- **EMG_STOP:** A high-priority state. If `STOP` is asserted, the elevator halts all movement and door operations immediately.

3.2 State Transition Logic

The movement logic is governed by the comparison:

$$\text{Next State} = \begin{cases} \text{GOING_UP} & \text{if } \text{curr_floor} < \text{latched_target} \\ \text{GOING_DOWN} & \text{if } \text{curr_floor} > \text{latched_target} \\ \text{OPEN_DOOR} & \text{if } \text{curr_floor} = \text{latched_target} \end{cases} \quad (1)$$

4 Testbench and Verification Process

The testbench (`tb_Elevator_Control`) provides a controlled environment to stress-test the elevator's logic.

4.1 Verification Components

1. **Clock Generator:** Produces a stable 100 MHz clock signal.
2. **Stimulus Process:** Mimics a user's interaction by setting the `CALL` vector and toggling the `STOP` button.
3. **UUT Mapping:** The Unit Under Test is instantiated and mapped to local signals for observation.

4.2 Test Scenarios

The verification process covers three primary sequences:

- **Ascension Test:** Calling floor 3 from floor 0 to verify `MOTOR_UP` and the `curr_floor` counter.
- **Sequential Move:** Moving from floor 3 to 5 to ensure the `latched_target` updates correctly from an `IDLE` state.
- **Safety Test:** Asserting `STOP` mid-transit to verify the `EMG_STOP` override logic.
- **Descension Test:** Moving from floor 5 to 2 to verify `MOTOR_DOWN` logic.

5 Key Technical Highlights of the Design

5.1 Incremental Floor Tracking

Unlike basic state machines that jump directly to a target, this design simulates real-world physics using a **Floor Tracking Register** (`curr_floor`). The elevator increments or decrements the floor index sequentially on each clock cycle, allowing for realistic integration with floor sensors in a physical implementation.

5.2 Input Latching Mechanism

To prevent erratic behavior due to input noise or changing user requests during transit, the system implements a **Target Latching** strategy.

The `CALL` input is only sampled and stored in `latched_target` when the machine is in the `IDLE` state.

This ensures the elevator completes its current journey before processing a new destination.

5.3 Global Emergency Override

The `EMG_STOP` logic is implemented with the highest priority in the combinatorial process. By placing the `if STOP = '1'` check outside of the FSM `case` statement, the design ensures that the elevator will halt immediately regardless of its current state (moving up, moving down, or opening doors).

5.4 Synchronous Door Timing

The `OPEN_DOOR` state utilizes a dedicated 4-bit synchronous timer. The door remains open for exactly 5 clock cycles:

$$T_{open} = 5 \times T_{period} \quad (2)$$

This predictable timing allows for safe passenger boarding and prevents the elevator from moving while the doors are active.

6 Conclusion

The design effectively simulates real-world elevator behavior. The use of a synchronous timer for door control and a dedicated emergency state ensures that the system is both functional and safe. Verification results confirm that the floor tracking and motor controls operate in perfect synchronization with the input requests.